Node JS Internals

JS has engine. That JS engine is the program inside a browser that reads, interprets and executes JS code.

SetInterval, fetch, async Promises, DOM ⇒ These are part of of Web APIs provided by the browser, and are not necessarily part of core JS.

Console is also provided by the browser to JS. JS is single threaded

LibUV gives us Event Loop, also gives event loop and thread pool.

V8 + c++ + libuv = Node JS

V8 ↑ chrome engine

c++ ↓

libuv ↓ event loop + thread pool + Non-blocking operations

Two ways to run a JS file

1. Run it in browser's console
2. Use "node <filename>.js" in your own terminal.

As soon as you run "node <filename>.js", it creates a node process ~~and~~ which has a main thread

Now what's happening ⇒

1. Init Project ↓
2. Top Level Code → Import Statements → Event Callbacks Register.

↓

Start Event loop

Top level code is assignment of variable, if else and console, loop
We dont run callback and functions.
After top level code, imports are done.
Then Event Callback are registered.
Now Event loops are started.

Random info ⇒ Ctrl+c, sends signals that ~~project~~ gracefully ends the process. A signal name "SIGINT" is run.
Also we do cleanup before ending the process. These are callback which are registers as the third step

⇒ Inside the Event loop, first "Expired callbacks are run"
& Then "IO Polling", IO Polling is a phase of event loop that checks if any asynchronous I/O operations have completed and executes their callbacks

Event loop ⇒ while True {
Expired Callback ()
IO polling ()
Set Immediate ()
close Callback))

check if anything unfinised task is left, if not continue and exit.

eg:

fs.readfile ('sample.txt', 'utf-8', (err, data) => {

. . . . . .

}

this is a callback

top level code

Blocking Code ⇒ should not be written.

Thread Pool is a pool of workers/pool of threads.

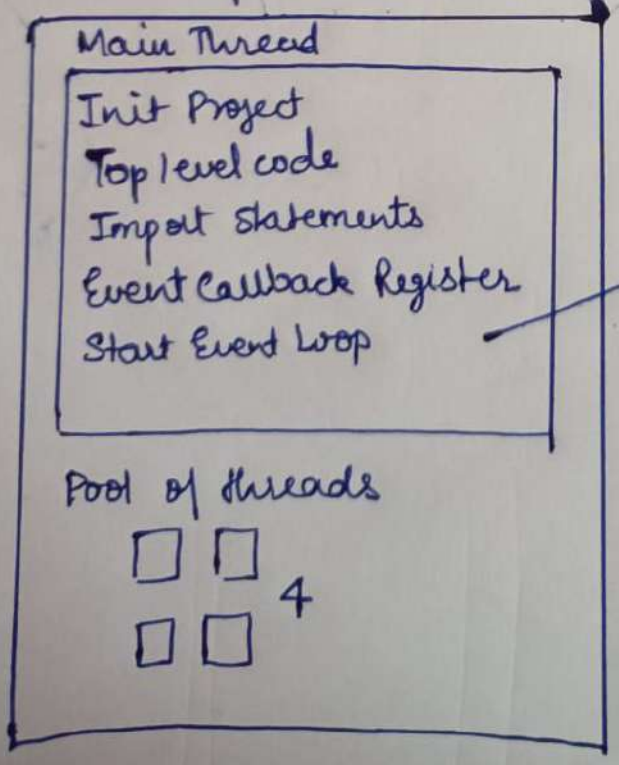
There a 4 threads. ~~JS main loop~~

JS main thread loop works independently.

If there is some CPU intensive task, Event loop delegates it to thread pool (in which it is assigned to one of its workers)

Io Polling asks the thread pool if task is done or not.

⇒ You can't decide which task is assigned to thread pool or event loop.



while True {

Expired Callbacks ()

Io Polling ()

Set Immediate ()

Close callback ()

checks if anything pending? No = Exit

Yes then continue